

12. A

执行前 3 句后, 栈 *st* 内的值为 *a, b*, 其中 *b* 为栈顶元素; 执行第 4 句后, 栈顶元素 *b* 出栈, *x* 的值为 *b*; 执行最后一句, 读取栈顶元素的值, *x* 的值为 *a*。

13. B

当 *n* 个不同元素进栈时, 出栈序列的个数为

$$\frac{1}{n+1}C_{2n}^n = \frac{1}{n+1} \frac{(2n)!}{n! \times n!} = \frac{6 \times 5 \times 4}{4 \times 3 \times 2 \times 1} = 5$$

考题中给出的 *n* 值不会很大, 可以根据栈的特点, 若 X_i 已经出栈, 则 X_i 前面的尚未出栈的元素一定逆置有序地出栈, 因此可采用例举方法。如 *a, b, c* 依次进栈的出栈序列有 *abc, acb, bac, bca, cba*。另外, 在一些考题中可能会问符合某个特定条件的出栈序列有多少种, 比如此题中的问以 *b* 开头的出栈序列有几种, 这种类型的题目一般都使用穷举法。

14. D

根据栈“先进后出”的特点, 且在进栈操作的同时允许出栈操作, 显然答案 D 中 *c* 最先出栈, 则此时栈内必定为 *a* 和 *b*, 但因为 *a* 先于 *b* 进栈, 所以要晚出栈。对于某个出栈的元素, 在它之前进栈却晚出栈的元素必定是按逆序出栈的, 其余答案均是可能出现的情况。

此题也可采用将各序列逐个代入的方法来确定是否有对应的进出栈序列 (类似下题)。

15. A

假设出栈序列为 *cd...*, 分析栈的操作序列: *a* 进栈, *b* 进栈, *c* 进栈, *c* 出栈, *d* 进栈, *d* 出栈, 此后只能是 *b* 出栈和 *a* 出栈一种情况, 因此出栈序列只有 *cdba*。

16. D

采用排除法, 选项 A, B, C 得到的出栈序列分别为 1243, 3241, 1324。由 1234 得到 1342 的进出栈序列为: 1 进, 1 出, 2 进, 3 进, 3 出, 4 进, 4 出, 2 出, 所以选 D。

17. D

第 *n* 个元素第一个出栈, 说明前 *n-1* 个元素都已经按顺序入栈, 由“先进后出”的特点可知, 此时的输出序列一定是输入序列的逆序, 所以答案选 D。

18. D

当第 *i* 个元素第一个出栈时, 则 *i* 之前的元素可以依次排在 *i* 之后出栈, 但剩余的元素可以在此时进栈并且也会排在 *i* 之前的元素出栈, 所以第 *j* 个出栈的元素是不确定的。

19. C

对于 A, 可能的顺序是 *a* 入, *a* 出, *b* 入, *b* 出, *c* 入, *c* 出, *d* 入, *d* 出。对于 B, 可能的顺序是 *a* 入, *b* 入, *c* 入, *c* 出, *b* 出, *d* 入, *d* 出, *a* 出。对于 D, 可能的顺序是 *a* 入, *a* 出, *b* 入, *c* 入, *c* 出, *b* 出, *d* 入, *d* 出。C 没有对应的序列。

【另解】若出栈序列的第一个元素为 *d*, 则出栈序列只能是 *dcba*。该思想通常也适用于出栈序列的局部分析: 如 12345 入栈, 问出栈序列 34152 是否正确? 如何分析? 若第一个出栈元素是 3, 则此时 12 必停留在栈中, 它们出栈的相对顺序只能是 21, 所以 34152 错误。

20. C

入栈序列是 $P_1, P_2, \dots, P_n$ 。由于 $P_3 = 1$, 即 P_1, P_2, P_3 连续入栈后, 第一个出栈元素是 P_3 , 说明 P_1, P_2 已经按序进栈, 根据先进后出的特点可知, P_2 必定在 P_1 之前出栈, 而第二个出栈元素是 2, 而此时 P_1 不是栈顶元素, 因此 P_1 的值不可能是 2。思考: 哪些 P_i 可能是 2?

21. A

假设 P_1 是 1, 进栈后立即出栈, P_2 是 2, 进栈后立即出栈, P_3 是 3, 进栈后立即出栈, 得到

的序列符合题意。假设 P_1 是 2, P_1 是 1, P_1 、 P_2 依次进栈后全部出栈, P_3 是 3, 进栈后立即出栈, 得到的序列符合题意。因此, P_1 既可能是 1, 又可能是 2。

22. C

逐个判断每个选项可能的入栈出栈顺序。对于 A, 可能的顺序是 1 入, 1 出, 2 入, 2 出, 3 入, 3 出, 4 入, 4 出。对于 B, 可能的顺序是 1 入, 2 入, 3 入, 3 出, 2 出, 4 入, 4 出, 1 出。对于 D, 可能的顺序是 1 入, 1 出, 2 入, 3 入, 3 出, 2 出, 4 入, 4 出。C 没有对应的序列, 因为当 4 在栈中时, 意味着前面的所有元素 (1, 2, 3) 都已在栈中或曾经入过栈, 此时若 4 第二个出栈, 即栈中还有两个元素, 且这两个元素是有序的 (对应入栈顺序), 只能为 (1, 2), (1, 3), (2, 3), 若是序列 (1, 2), 则 3 已在 p_1 位置出栈, 不可能再在 p_4 位置出栈, 若是 (1, 3) 和 (2, 3) 这种情况中的任意一种, 则 3 一定是下一个出栈元素, 即 p_3 一定是 3, 所以 p_4 不可能是 3。

【另解】对于 C, p_2 为最后一个入栈元素 4, 则只有 p_1 或 p_3 出栈的元素有可能为 3 (请读者分两种情况自行思考), 而 p_4 绝不可能为 3。读者在解答此类题时, 一定要注意出栈序列中的“最后一个入栈元素”, 这样可以节省答题的时间。

23. C

标识符只能以英文字母或下画线开头, 而不能以数字开头。于上, 由 n、1、_ 三个字符组合成的标识符有 n1_、n_1、_1n 和 _n1 四种。第一种: n 进栈再出栈, 1 进栈再出栈, _ 进栈再出栈。第二种: n 进栈再出栈, 1 进栈, _ 进栈, _ 出栈, 1 出栈。第三种: n 进栈, 1 进栈, _ 进栈, _ 出栈, 1 出栈, n 出栈。而根据栈的操作特性, _n1 这种情况不可能出现。

24. B

上溢是指存储器满, 还往里写; 下溢是指存储器空, 还往外读。为了解决上溢, 可给栈分配很大的存储空间, 但这样又会造成存储空间的浪费。共享栈的提出就是为了在解决上溢的基础上节省存储空间, 将两个栈放在同一段更大的存储空间内, 这样, 当一个栈的元素增加时, 可使用另一个栈的空闲空间, 从而降低发生上溢的可能性。

25. A

这种情况就是前面我们所描述的, 详细内容请参见本节考点精析部分对共享栈的讲解。另外, 读者可以思考若 $top1$ 的初值为 0, $top2$ 的初值为 $n-1$ 时栈满的条件。

注 意

栈顶、队头与队尾的指针的定义是不唯一的, 做题时务必仔细审题和思考。

26. C

时刻注意栈的特点是先进后出, 下表是出入栈的详细过程。

序号	说明	栈内	栈外	序号	说明	栈内	栈外
1	a 入栈	a		8	e 入栈	ae	bdc
2	b 入栈	ab		9	f 入栈	aef	bdc
3	b 出栈	a	b	10	f 出栈	ae	bdcf
4	c 入栈	ac	b	11	e 出栈	a	bdcfe
5	d 入栈	acd	b	12	a 出栈		bdcfea
6	d 出栈	ac	bd	13	g 入栈	g	bdcfea
7	c 出栈	a	bdc	14	g 出栈		bdcfeag

栈内的最大深度为 3，所以栈 S 的容量至少是 3。

【另解】因为元素的出队顺序和入队顺序相同，所以元素的出栈顺序就是 b, d, c, f, e, a, g ，因此元素的出入栈次序为 $\text{Push}(S, a), \text{Push}(S, b), \text{Pop}(S, b), \text{Push}(S, c), \text{Push}(S, d), \text{Pop}(S, d), \text{Pop}(S, c), \text{Push}(S, e), \text{Push}(S, f), \text{Pop}(S, f), \text{Pop}(S, e), \text{Pop}(S, a), \text{Push}(S, g), \text{Pop}(S, g)$ 。初始所需容量为 0，每做一次 Push 操作，容量加 1；每做一次 Pop 操作，容量减 1，记录的容量最大值为 3。

27. D

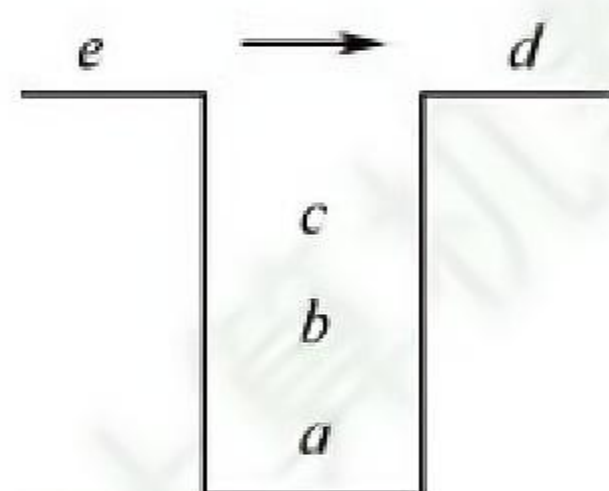
选项 A 由 a 进， b 进， c 进， d 进， d 出， c 出， e 进， e 出， b 出， f 进， f 出， a 出得到；选项 B 由 a 进， b 进， c 进， c 出， b 出， d 进， d 出， a 出， e 进， e 出， f 进， f 出得到；选项 C 由 a 进， b 进， b 出， c 进， c 出， a 出， d 进， e 进， e 出， f 进， f 出， d 出得到；选项 D 由 a 进， a 出， b 进， c 进， d 进， e 进， f 进， f 出， e 出， d 出， c 出， b 出得到，但题意要求不允许连续 3 次退栈操作，选项 D 不符。

【另解】先进栈的元素后出栈，进栈顺序为 a, b, c, d, e, f ，所以连续出栈时的子序列必然是按字母表逆序的，若出栈序列中出现了长度大于或等于 3 的连续逆序子序列，则为所选序列。

28. B

d 第一个出栈，则 c, b, a 出栈的相对顺序是确定的，出栈顺序必为 $d_c_b_a_$ ， e 的顺序不定，在任意一个“ $_$ ”上都有可能。

【另解】 d 首先出栈，则 abc 停留在栈中，此时栈的状态如下图所示。



此时可以有如下 4 种操作：① e 进栈后出栈，则出栈序列为 $decba$ ；② c 出栈， e 进栈后出栈，出栈序列为 $dceba$ ；③ cb 出栈， e 进栈后出栈，出栈序列为 $dcbea$ ；④ cba 出栈， e 进栈后出栈，出栈序列为 $dcbae$ 。思路和上面其实一样。

29. C

显然，3 之后的 $4, 5, \dots, n$ 都是 P_3 可取的数（一直进栈直到该数入栈后马上出栈）。接下来分析 1 和 2 是否可取： P_1 可以是 3 之前入栈的数（可能是 1 或 2），也可以是 4，当 $P_1=1$ 时， P_3 可取 2；当 $P_1=2$ 时， P_3 可取 1。因此， p_3 可能取除 3 外的所有数，个数为 $n-1$ 。

30. D

按题意，出入栈操作的过程如下：

操作	栈内元素	出栈元素
Push	a	
Push	$a\ b$	
Pop	a	b
Push	$a\ c$	
Pop	a	c
Push	$a\ d$	
Push	$a\ d\ e$	
Pop	$a\ d$	e

因此，出栈序列为 b, c, e 。

31. D

通过模拟出入栈操作，可以判断入栈序列 in 和出栈序列 out 是否合法。因此，已知 in 序列可以判断 out 序列是否为可能的出栈序列；已知 out 序列也可以判断 in 序列是否为可能的入栈序列，A 和 B 错误。若每个元素入栈后立即出栈，则 in 序列和 out 序列相同，C 错误。若所有元素都入栈后才依次出栈，则 in 序列和 out 序列互为倒序，D 正确。

3.2.6 答案与解析

一、单项选择题

01. D

栈和队列的逻辑结构都是线性结构，都可以采用顺序存储或链式存储，C 显然也错误。只有 D 才是栈和队列的本质区别，限定表中插入和删除操作位置的不同。

02. B

队列“先进先出”的特性表现在：先进队列的元素先出队列，后进队列的元素后出队列，进队列对应的是插入操作，出队列对应的是删除操作。I 和 IV 均正确。

03. D

删除队头元素即出队，是队列的基本操作之一，所以选 D。

04. B

队列的入队顺序和出队顺序是一致的，这是和栈不同的。

05. D

数组下标范围 $0 \sim n$ ，因此数组容量为 $n+1$ 。循环队列中元素入队的操作是 $\text{rear} = (\text{rear} + 1) \bmod \text{maxsize}$ ，题中 $\text{maxsize} = n+1$ 。因此入队操作应为 $\text{rear} = (\text{rear} + 1) \bmod (n+1)$ 。

06. C

队列的长度为 $(\text{rear} - \text{front} + \text{maxsize}) \% \text{maxsize} = (\text{rear} - \text{front} + 21) \% 21 = 16$ 。这种情况和 front 指向当前元素， rear 指向队尾元素的下一个元素的计算是相同的。

注意

数组 $A[n]$ 的下标范围为 $0 \sim n-1$ 。若写成 $A[0 \dots n]$ ，则说明下标范围为 $0 \sim n$ 。

07. B

循环队列中，每删除一个元素，队首指针 $\text{front} = (\text{front} + 1) \% 6$ ，每插入一个元素，队尾指针 $\text{rear} = (\text{rear} + 1) \% 6$ 。上述操作后， $\text{front} = 0$ ， $\text{rear} = 3$ 。

08. D

当队列中只有一个元素时， front 指向该元素的前一个位置， rear 指向该元素，因此当队列为空时，队首指针等于队尾指针，这样第一个元素进队后，才能符合题目要求。

09. C

既然不能附加任何其他数据成员，只能采用牺牲一个存储单元的方法来区分是队空还是队满，约定以“队列头指针在队尾指针的下一位置作为队满的标志”，因此选 C。选项 A 是判断队列是否空的条件，选项 B 和 D 都是干扰项。

注意

对于这类具体问题，举一些特例判断往往比直接思考问题能更快得到答案。

10. A

若用 $(\text{rear} + 1) \% (n + 1) == \text{front}$ 作为队满标志，则说明题目采用了牺牲一个存储单元的方

法来区分队空和队满，因此可用 $\text{front} == \text{rear}$ 作为队空标志。

11. D

虽然链式队列采用动态分配方式，但其长度也受内存空间的限制，不能无限制增长。顺序队列和链式队列的进队和出队时间均为 $O(1)$ 。顺序队列和链式队列都可以进行顺序访问。对于顺序队列，可通过队头指针和队尾指针计算队列中的元素个数，而链式队列则不能。

12. B

因为队列需在双端进行操作，所以选项 C 和 D 的链表显然不太适合链队。对于 A，链表在完成进队和出队后还要修改为循环的，对于队列来讲这是多余的（画蛇添足）。对于 B，因为有首指针，所以适合删除首结点；因为有尾指针，所以适合在其后插入结点。

13. A

因为非循环双链表只带队首指针，所以在执行入队操作时需要修改队尾结点的指针域，而查找队尾结点需要 $O(n)$ 的时间。B、C 和 D 均可在 $O(1)$ 的时间内找到队首和队尾。

14. A

因为在队头做出队操作，为便于删除队头元素，所以总是选择链头作为队头。

15. D

队列用链式存储时，删除元素从表头删除，通常仅需修改头指针，但若队列中仅有一个元素，则尾指针也需要被修改，当仅有一个元素时，删除后队列为空，需修改尾指针为 $\text{rear} = \text{front}$ 。

16. D

插入操作时，先将结点 x 插入到链表尾部，再让 rear 指向这个结点 x 。C 的做法不够严密，因为是队尾，所以队尾 $x \rightarrow \text{next}$ 必须置为空。

17. A

依题意，进队操作是在队尾进行，即链表表头。题中已明确说明链表只设头指针，也即没有头结点和尾指针，进队后，循环单链表必须保持循环的性质，在只带头指针的循环单链表中寻找表尾结点的时间复杂度为 $O(n)$ ，所以进队的时间复杂度为 $O(n)$ 。

18. B

此类题可对各选项进行模拟，假设队列为 Q_1 和 Q_2 。对于 A，元素依次入队 Q_1 ，然后依次出队。对于 B，5 最先出队，只可能是 1, 2, 3, 4 入队 Q_1 ，5 入队 Q_2 ，然后 5 出队，只能得到 5, 1, 2, 3, 4。对于 C，1, 2, 5 入队 Q_1 ，3, 4 入队 Q_2 ，然后按要求出队。D 的分析同 C。

19. C

使用排除法。先看可由输入受限的双端队列产生的序列：设右端输入受限，1, 2, 3, 4 依次左入，则依次左出可得 4, 3, 2, 1，排除 A；左出、右出、左出、左出可得到 4, 1, 3, 2，排除 B；再看可由输出受限的双端队列产生的序列：设右端输出受限，1, 2, 3, 4 依次左入、左入、右入、左入，依次左出可得到 4, 2, 1, 3，排除 D。

20. C

本题的队列实际上是一个输出受限的双端队列，如图 3.11 所示。A 操作： a 左入（或右入）、 b 左入、 c 右入、 d 右入、 e 右入。B 操作： a 左入（或右入）、 b 左入、 c 右入、 d 左入、 e 右入。D 操作： a 左入（或右入）、 b 左入、 c 左入、 d 右入、 e 左入。C 操作： a 左入（或右入）、 b 右入、因 d 未出，此时只能进队， c 怎么进都不可能在 b 和 a 之间。

【另解】初始时队列为空，第 1 个元素 a 左入（或右入）后，第 2 个元素 b 无论是左入还是右入都必与 a 相邻，而选项 C 中 a 与 b 不相邻，不合题意。

21. B

根据题意, 第一个元素进入队列后存储在 $A[0]$ 处, 此时 $front$ 和 $rear$ 值都为 0。入队时因为要执行 $(rear+1)\%n$ 操作, 所以若入队后指针指向 0, 则 $rear$ 初值为 $n-1$, 而因为第一个元素在 $A[0]$ 中, 插入操作只改变 $rear$ 指针, 所以 $front$ 为 0 不变。

注 意

①循环队列是指顺序存储的队列, 而不是指逻辑上的循环, 如循环单链表表示的队列不能称为循环队列。② $front$ 和 $rear$ 的初值并不是固定的。

22. A

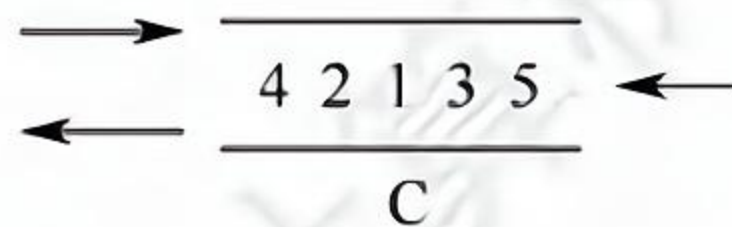
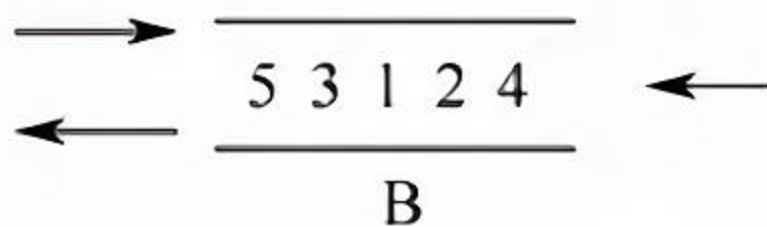
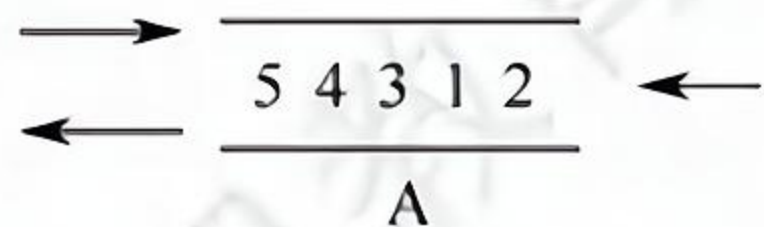
$end1$ 指向队头元素, 可知出队操作是先从 $A[end1]$ 读数, 然后 $end1$ 再加 1。 $end2$ 指向队尾元素的后一个位置, 可知入队操作是先存数到 $A[end2]$, 然后 $end2$ 再加 1。若用 $A[0]$ 存储第一个元素, 队列初始时, 入队操作是先把数据放到 $A[0]$ 中, 然后 $end2$ 自增, 即可知 $end2$ 初值为 0; 而 $end1$ 指向的是队头元素, 队头元素在数组 A 中的下标为 0, 所以得知 $end1$ 的初值也为 0, 可知队空条件为 $end1==end2$; 然后考虑队列满时, 因为队列最多能容纳 $M-1$ 个元素, 假设队列存储在下标为 0 到 $M-2$ 的 $M-1$ 个区域, 队头为 $A[0]$, 队尾为 $A[M-2]$, 此时队列满, 考虑在这种情况下 $end1$ 和 $end2$ 的状态, $end1$ 指向队头元素, 可知 $end1=0$, $end2$ 指向队尾元素的后一个位置, 可知 $end2=M-2+1=M-1$, 所以队满的条件为 $end1==(end2+1)\%M$ 。

23. C

选项 A 的操作顺序为①①②②①①③③。选项 B 的操作顺序为②①①①①①③。选项 D 的操作顺序为②②②②②①③③③③③。对于 C: 首先输出 3, 说明 1 和 2 必须先依次入栈, 而此后 2 肯定比 1 先输出, 因此无法得到 1, 2 的输出顺序。

24. D

假设队列左端允许入队和出队, 右端只能入队。对于 A, 依次从右端入队 1, 2, 再从左端入队 3, 4, 5。对于 B, 从右端入队 1, 2, 然后从左端入队 3, 再从右端入队 4, 最后从左端入队 5。对于 C, 从左端入队 1, 2, 然后从右端入队 3, 再从左端入队 4, 最后从右端入队 5。无法验证 D 的序列。



3.3.7 答案与解析

一、单项选择题

01. D

缓冲区是用队列实现的，A、B、C 都是栈的典型应用。

02. B

后缀表达式中，每个计算符号均直接位于其两个操作数的后面，按照这样的方式逐步根据计算的优先级将每个计算式进行变换，即可得到后缀表达式。

另解：将两个直接操作数用括号括起来，再将操作符提到括号后，最后去掉括号。例如，对于 $(①(②a*(③b+c))-d)$ ，提出操作符并去掉括号后，可得后缀表达式为 $abc+*d-$ 。

学完第 5 章后，可将表达式画成二叉树的形式，再用后序遍历即可求得后缀表达式。

03. D

FIFO 页面替换算法用到了队列。其余的都只用到了栈。

04. B

利用栈求表达式的值时，可以分别设立运算符栈和运算数栈，其原理不变。选项 B 中 A 入栈，B 入栈，计算得 R1，C 入栈，计算得 R2，D 入栈，计算得 R3，由此得栈深为 2。A、C、D 依次计算得栈深为 4、3、3。因此选 B。

技巧：根据算符优先级，统计已依次进栈，但还没有参与计算的运算符的个数。以选项 C 为例，当 “(” “A” “-” 入栈时，“(” 和 “-” 还没有参与运算，此时运算符栈大小为 2，“B” 和 “*” 入栈时运算符大小为 3，“C” 入栈时 “B*C” 运算，此时运算符栈大小为 2，以此类推。

05. B

栈与递归有着紧密的联系。递归模型包括递归出口和递归体两个方面。递归出口是递归算法的出口，即终止递归的条件。递归体是一个递推的关系式。根据题意有

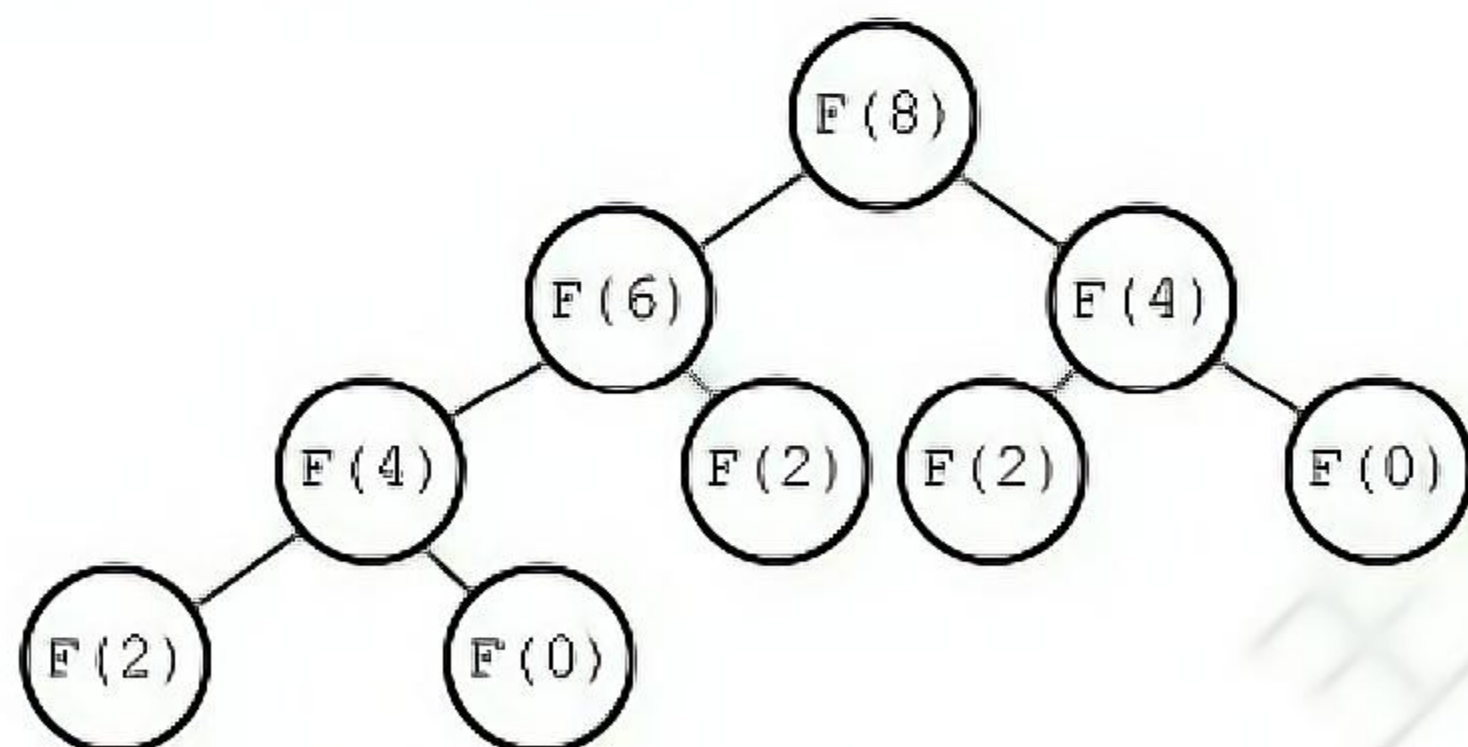
$$f(0)=2;$$

$$f(1)=1*f(0)=2;$$

$$f(f(1))=f(2)=2*f(1)=4。$$

06. C

计算 $F(8)$ 的递归调用树如下图所示：



由图可知，递归函数 $F()$ 调用的次数为 9。

07. C

首先，执行内层参数 $\text{func}(5)=\text{func}(3)+\text{func}(1)=4$ ，共执行 3 次 func 函数。然后，执行 $\text{func}(\text{func}(5))=\text{func}(4)=\text{func}(2)+\text{func}(0)=4$ ，因此第 4 个被执行的 func 函数是 $\text{func}(4)$ 。也可以采用画出递归调用树的方式，即某个函数的执行次序等于其在递归调用树的层次遍历中的次序。

08. B

通常情况下，递归算法在计算机实际执行的过程中包含很多的重复计算，所以效率会低。

09. C

调用函数时，系统会为调用者构造一个由参数表和返回地址组成的活动记录，并将记录压入系统提供的栈中，若被调用函数有局部变量，也要压入栈中。

10. B

本题涉及第5章和第6章的内容，图的广度优先搜索类似于树的层序遍历，都要借助于队列。

11. A

使用栈可以模拟递归的过程，以此来消除递归，但对于单向递归和尾递归而言，可以用迭代的方式来消除递归，A正确。不同的进栈和出栈组合操作，会产生许多不同的输出序列，B错误。通常使用栈来处理函数或过程调用，C错误。队列和栈都是操作受限的线性表，但只有队列允许在表的两端进行运算，而栈只允许在栈顶方向进行操作，D错误。

12. B

在提取数据时必须保持原来数据的顺序，所以缓冲区的特性是先进先出。

13. A

在中缀表达式转后缀表达式的过程中，扫描到操作数时直接输出，扫描到操作符时根据其优先级进行相应的出入栈操作。有几点需要注意：①若遇到界限符“（”，则直接入栈；②若遇到界限符“）”，则不入栈，且会依次弹出栈顶运算符，直到遇到“（”为止，并删除“（”；③若当前运算符的优先级高于栈顶运算符或者遇到栈顶为“（”，则直接入栈；④若当前运算符的优先级低于或等于栈顶运算符，则依次弹出，直到遇到一个优先级高于它的运算符或者遇到“（”为止，之后将当前运算符入栈。

求栈中操作符的最大个数时，为简单起见，可省略对操作数的处理。

步	待处理运算符	栈	扫描运算符	动作
1	$+b-a*((c+d)/e-f)+g$		+	+入栈
2	$-a*((c+d)/e-f)+g$	+	-	-优先级等于栈顶+，弹出+，-入栈
3	$*((c+d)/e-f)+g$	-	*	*优先级高于栈顶-，*入栈
4	$((c+d)/e-f)+g$	-*	(	(直接入栈
5	$(c+d)/e-f)+g$	-*(	(	(直接入栈
6	$+d)/e-f)+g$	-*((	+	栈顶为(，+直接入栈
7	$) / e - f) + g$	-*(((+	)	遇到)，弹出+，删除(
8	$/ e - f) + g$	-*(	/	栈顶为(，/直接入栈
10	$-f)+g$	-*(/	-	-优先级低于栈顶，弹出-，-入栈
11	$) + g$	-*(/-	)	遇到)，弹出-和/，删除(
12	$+g$	-*	+	+优先级低于栈顶*，等于-，依次弹出+和-；+入栈
13		+		

由上述过程可知，栈中操作符的最大个数为5。

14. B

中缀表达式 $a/b+(c*d-e*f)/g$ 转换为后缀表达式的过程如下：

步	待处理序列	栈	后缀表达式	扫描项	动作
1	$a/b+(c*d-e*f)/g$			a	a 加入后缀表达式
2	$/b+(c*d-e*f)/g$		a	/	/入栈
3	$b+(c*d-e*f)/g$	/	a	b	b 加入后缀表达式

(续表)

步	待处理序列	栈	后缀表达式	扫描项	动作
4	$+(c*d-e*f)/g$	/	Ab	+	+优先级低于栈顶的/, 弹出/, +入栈
5	$(c*d-e*f)/g$	+	ab/	(	(入栈
6	$c*d-e*f)/g$	+(	ab/	c	c 加入后缀表达式
7	$*d-e*f)/g$	+(	ab/c	*	栈顶为(, *入栈
8	$d-e*f)/g$	+(*	ab/c	d	d 加入后缀表达式
9	$-e*f)/g$	+(*	ab/cd	-	-优先级低于栈顶的*, 弹出*, -入栈
10	$e*f)/g$	+(-	ab/cd*	e	e 加入后缀表达式
11	$*f)/g$	+(-	ab/cd*e	*	*优先级高于栈顶的-, *入栈
12	$f)/g$	+(-*	ab/cd*e	f	f 加入后缀表达式
13	$) /g$	+(-*	ab/cd*ef	)	遇到), 依次弹出*, -加入表达式, 删除(
14	$/g$	+	ab/cd*ef*-	/	/优先级高于栈顶的+, /入栈
15	g	+/	ab/cd*ef*-	g	g 加入后缀表达式
16		+/	ab/cd*ef*-g		扫描完毕, 运算符依次弹出加入表达式
17			ab/cd*ef*-g/+		完成

由此可知, 当扫描到 f 时, 栈中的元素依次是 $+(-*$ 。

【另解】采用手算方法, 得出中缀式 $a/b+(c*d-e*f)/g$ 对应的后缀式为 $ab/cd*ef*-g/+$ 。中缀表达式转后缀表达式时, 操作数都直接输出, 因此操作数的顺序是固定的。扫描到 f 时, 在后缀表达式 f 后面的运算符要么还未入栈, 要么还在栈中, 需要结合中缀式来判断, f 后面依次出栈的运算符为 $*-/+$, $/$ 在中缀表达式 f 之后, 此时还未入栈, 因此栈中的运算符 (从栈底到栈顶) 为 $+-*$; 此外, 已入栈的界限符 (此时还未消解, 因此 $($ 也还在栈中, 栈中的元素依次是 $+(-*$ 。

15. A

递归调用函数时, 在系统栈中保存的函数信息需满足先进后出的特点, 依次调用了 $main(), S(1), S(0)$, 所以栈底到栈顶的信息依次是 $main(), S(1), S(0)$ 。

注意

在递归中, 系统为每一层的返回点、局部变量、传入实参等开辟了递归工作栈来存储。

16. C

根据题意: 入队顺序为 8, 4, 2, 5, 3, 9, 1, 6, 7, 出队顺序为 1~9。入口和出口之间有多个队列 (n 条轨道), 且每个队列 (轨道) 可容纳多个元素 (多列列车), 为便于区分, 队列用字母编号。分析如下: 显然先入队的元素必须小于后入队的元素 (否则, 若 8 和 4 入同一个队列, 8 在 4 前面, 则出队时也只能 8 在 4 前面), 这样 8 入队列 A, 4 入队列 B, 2 入队列 C, 5 入队列 B (按照前述原则 “大的元素在小的元素后面” 也可将 5 入队列 C, 但这时剩下的元素 3 就必须放入一个新的队列中, 无法确保 “至少”), 3 入队列 C, 9 入队列 A, 这时共占了 3 个队列, 后面还有元素 1, 直接再用一个新的队列 D, 1 从队列 D 出队后, 剩下的元素 6 和 7 或入队列 B, 或入队列 C。综上, 共占用了 4 个队列。当然还有其他的入队、出队情况, 请读者自己推演, 但要确保满足: ①队列中后面的元素大于前面的元素; ②确保占用最少 (即满足题意中 “至少”) 的队列。

17. C

I 的反例: 计算斐波拉契数列迭代实现只需要一个循环即可实现。III 的反例: 入栈序列为 1, 2,

进行 Push, Push, Pop, Pop 操作, 出栈次序为 2、1; 进行 Push, Pop, Push, Pop 操作, 出栈次序为 1, 2。IV, 栈是一种受限的线性表, 只允许在一端进行操作。II 正确。

18. B

第一次调用: ①从 s1 中弹出 2 和 3; ②从 s2 中弹出+; ③执行 $3+2=5$; ④将 5 压入 s1 中, 第一次调用结束后 s1 中剩余 5、8、5 (5 在栈顶), s2 中剩余*、- (-在栈顶)。第二次调用: ①从 s1 中弹出 5 和 8; ②从 s2 中弹出-; ③执行 $8-5=3$; ④将 3 压入 s1 中, 第二次调用结束后 s1 中剩余 5、3 (3 在栈顶), s2 中剩余*。第三次调用: ①从 s1 中弹出 3 和 5; ②从 s2 中弹出*; ③执行 $5*3=15$; ④将 15 压入 s1 中, 第三次调用结束后 s1 中仅剩余 15, s2 为空。

3.4.6 答案与解析

单项选择题

01. D

特殊矩阵中含有很多相同元素或零元素，所以可采用压缩存储，以节省存储空间。

02. C

只需存储其上三角或下三角部分（含对角线），元素个数为 $n + (n-1) + \dots + 1 = n(n+1)/2$ 。

03. A

此题要注意 3 个细节：矩阵的最小下标为 0；数组下标也是从 0 开始的；矩阵按行优先存在数组中。注意到此三点，答案不难得到为 A。此外，本类题建议采用特殊值代入法求解，例如， $A[1][1]$ 对应的下标应为 2，代入后只有 A 满足条件。

技巧：对于特殊三角矩阵压缩存储的题，心中应有“平移”搬动的思想，并结合草图，这样会比较形象，在计算时再注意矩阵和数组的起始下标，就不容易出错。

04. D

二维数组计算地址（按行优先顺序）的公式为

$$\text{LOC}(i, j) = \text{LOC}(0, 0) + (i \times m + j) \times L$$

其中， $\text{LOC}(0, 0) = \text{SA}$ ，是数组存放的首地址； $L = 3$ 是每个数组元素的长度； $m = 9 - 0 + 1 = 10$ 是数组的列数。因此有 $\text{LOC}(8, 5) = \text{SA} + (8 \times 10 + 5) \times 3 = \text{SA} + 255$ ，所以选 D。

05. A

本题未直接给出数组 A 的行数和列数，因此需要根据题目中的信息来推理。因为该二维数组按行优先存储，且 $A[3][3]$ 的存储地址为 446，所以 $A[3][1]$ 的存储地址为 444，又 $A[1][1]$ 的存储地址为 420，显然 $A[1][1]$ 和 $A[3][1]$ 正好相差 2 行，所以该矩阵的列数为 12。而 $A[5][3]$ 和 $A[3][3]$ 正好相差 2 行， $A[5][5]$ 和 $A[5][3]$ 又相差 2 个元素，所以 $A[5][5]$ 的存储地址

是 $446 + 24 + 2 = 472$ 。

06. B

对于三对角矩阵, 将 $A[1 \dots n][1 \dots n]$ 压缩至 $B[1 \dots 3n-2]$ 时, a_{ij} 与 b_k 的对应关系为 $k = 2i + j - 2$ 。则 A 中的元素 $A[66][65]$ 在数组 B 中的位置 k 为 $2 \times 66 + 65 - 2 = 195$ 。

07. C

按列优先存储, 所以元素 a_{ij} 前面有 $j-1$ 列, 共有 $1+2+3+\dots+j-1 = j(j-1)/2$ 个元素, 元素 a_{ij} 在第 j 列上是第 i 个元素, 数组 B 的下标是从 1 开始, 因此 $k = j(j-1)/2 + i$ 。

08. B

按列优先存储, 所以元素 a_{ij} 之前有 $j-1$ 列, 共有 $n+(n-1)+\dots+(n-j+2) = (j-1)(2n-j+2)/2$ 个元素, 元素 a_{ij} 是第 j 列上第 $i-j+1$ 个元素, 数组 B 的下标从 1 开始, $k = (j-1)(2n-j+2)/2 + i-j+1$ 。

09. B

稀疏矩阵通常采用三元组来压缩存储, 存储矩阵元素的行列下标和相应的值, 因此不能根据矩阵元素的行列下标快速定位矩阵元素, 失去了随机存取的特性。

10. D

在三对角矩阵中, 第 1 行和最后 1 行只有 2 个非零元, 其余各行均有 3 个非零元。稀疏矩阵的特点是矩阵中非零元的个数较少。

11. B

三对角矩阵如下所示。

$$\begin{bmatrix} a_{1,1} & a_{1,2} & & & & \\ a_{2,1} & a_{2,2} & a_{2,3} & & & 0 \\ & a_{3,2} & a_{3,3} & a_{3,4} & & \\ & & \ddots & \ddots & \ddots & \\ & 0 & & a_{n-1,n-2} & a_{n-1,n-1} & a_{n-1,n} \\ & & & & a_{n,n-1} & a_{n,n} \end{bmatrix}$$

采用压缩存储, 将 3 条对角线上的元素按行优先方式存放在一维数组 B 中, 且 $a_{1,1}$ 存放于 $B[0]$ 中, 其存储形式如下所示:

$a_{1,1}$	$a_{1,2}$	$a_{2,1}$	$a_{2,2}$	$a_{2,3}$	$\dots$	$a_{n-1,n}$	$a_{n,n-1}$	$a_{n,n}$
-----------	-----------	-----------	-----------	-----------	---------	-------------	-------------	-----------

可以计算矩阵 A 中 3 条对角线上的元素 a_{ij} ($1 \leq i, j \leq n, |i-j| \leq 1$) 在一维数组 B 中存放的下标为 $k = 2i + j - 3$, 公式很难记忆, 我们通常采用解法 2。

解法 1: 针对该题, 仅需将数字逐一代入公式: $k = 2 \times 30 + 30 - 3 = 87$, 结果为 87。

解法 2: 观察上图的三对角矩阵不难发现, 第一行有两个元素, 剩下的在元素 $m_{30,30}$ 所在行之前的 28 行 (注意下标 $1 \leq i, j \leq 100$) 中, 每行都有 3 个元素, 而 $m_{30,30}$ 之前仅有一个元素 $m_{30,29}$, 不难发现元素 $m_{30,30}$ 在数组 N 中的下标是 $2 + 28 \times 3 + 2 - 1 = 87$ 。

注 意

矩阵和数组的下标从 0 或 1 开始 (如矩阵可能从 $a_{0,0}$ 或 $a_{1,1}$ 开始, 数组可能从 $B[0]$ 或 $B[1]$ 开始), 这时就需要适时调整计算方法 (方法无非是多计算 1 或少计算 1 的问题)。

12. A

三元组表的结点存储了行 (row)、列 (col)、值 (value) 三种信息, 是主要用来存储稀疏矩阵

的一种数据结构。十字链表将行单链表和列单链表结合起来存储稀疏矩阵。邻接矩阵空间复杂度达 $O(n^2)$ ，不适合于存储稀疏矩阵。二叉链表又名左孩子右兄弟表示法，可用于表示树或森林。

13. A

在 C 语言中，数组 N 的下标从 0 开始。第一个元素 $m_{1,1}$ 对应存入 n_0 ，矩阵 M 的第一行有 12 个元素，第二行有 11 个，第三行有 10 个，第四行有 9 个，第五行有 8 个，所以 $m_{6,6}$ 是第 $12+11+10+9+8+1=51$ 个元素，下标应为 50。

14. C

上三角矩阵按列优先存储，先存储只有 1 个元素的第一列，再存储有 2 个元素的第二列，以此类推。 $m_{7,2}$ 位于左下角，对应右上角的元素为 $m_{2,7}$ ，在 $m_{2,7}$ 之前存有

第 1 列：1

第 2 列：2

.....

第 6 列：6

第 7 列：1

前面共存储有 $1+2+3+4+5+6+1=22$ 个元素（数组下标范围为 0~21），注意数组下标从 0 开始，所以 $m_{2,7}$ 在数组 N 中的下标为 22，即 $m_{7,2}$ 在数组 N 中的下标为 22。

15. B

二维数组 A 按行优先存储，每个元素占用 1 个存储单元，由 $A[0][0]$ 和 $A[3][3]$ 的存储地址可知 $A[3][3]$ 是二维数组 A 中的第 121 个元素，假设二维数组 A 的每行有 n 个元素，则 $n \times 3 + 4 = 121$ ，求得 $n = 39$ ，所以元素 $A[5][5]$ 的存储地址为 $100 + 39 \times 5 + 6 - 1 = 300$ 。

16. A

用三元组表存储结构存储稀疏矩阵 M 时，每个非零元素都由三元组（行标、列标、关键字值）组成。但是，仅通过三元组表中的元素无法判断稀疏矩阵 M 的大小，因此还要保存 M 的行数和列数。此外，还可以保存 M 的非零元素个数。例如，如下两个稀疏矩阵的三元组表是相同的，若不保存行数和列数，则无法判断两个稀疏矩阵的大小。

0	2	0	0
0	0	5	0
0	0	0	0
0	0	0	0

0	2	0
0	0	5
0	0	0